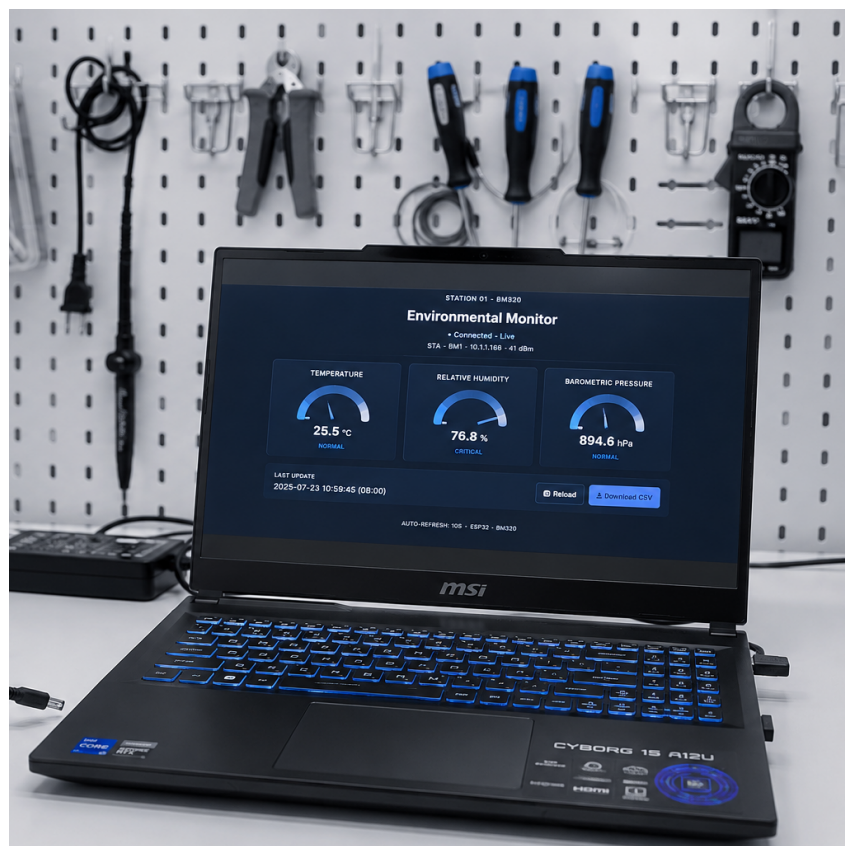


Environmental Monitor & Data Logger

ESP32 + BME280 Local HTTP Dashboard



*Dana Natsheh
Cyber Robot
IoT*

1. Executive Summary

This report documents the design, implementation, and debugging of a self-contained environmental monitoring station built around an ESP32 development board and a BME280 digital sensor. The device samples temperature, relative humidity, and barometric pressure over I2C, serves a live, auto-refreshing web dashboard directly from the ESP32 (no phone app or cloud service required), and appends every reading to a CSV log file stored on the device's internal flash filesystem, downloadable on demand from the browser.

Beyond the assignment's core requirements, the firmware includes several deliberate reliability additions: automatic WiFi fallback from station mode to a self-hosted access point so the dashboard is never unreachable, NTP time synchronization for real-world CSV timestamps with a graceful offline fallback, log-file size rotation to prevent unbounded flash growth, and – critically – persistence of the last known sensor reading in non-volatile storage (NVS), so a power loss or reset does not leave the dashboard blank until the next sensor sample arrives.

The build surfaced a representative set of embedded/full-stack debugging challenges, spanning toolchain differences between the Arduino IDE and PlatformIO, a network-layer connectivity failure that had to be isolated from a firmware failure, and a physical-world calibration issue (local barometric baseline) that only became visible once real data was logged. Each is documented in detail in Section , alongside the reasoning used to diagnose it – the debugging methodology is treated as being just as much a deliverable as the working firmware itself.

2. Hardware Architecture & Pin Mapping

The project uses a single ESP32 DevKit development board and one BME280 breakout module communicating over I2C. The BME280 is a 3.3 V-logic device and is powered directly from the ESP32's 3.3 V rail – it is not 5 V tolerant, and connecting it to a 5 V rail is a common first-time wiring mistake that this build deliberately avoided.

Peripheral	Peripheral Pin	ESP32 GPIO	Electrical Logic / Notes
BME280 Sensor	VCC	3.3V	Logic supply – not 5V tolerant
BME280 Sensor	GND	GND	Common ground reference
BME280 Sensor	SCL	GPIO 22	I2C clock (<code>I2C_SCL_PIN</code>)
BME280 Sensor	SDA	GPIO 21	I2C data (<code>I2C_SDA_PIN</code>)

Table 1: ESP32 pin mapping for the BME280 environmental sensor.

I2C address handling

BME280 breakout boards ship at either I2C address `0x76` or `0x77`, depending on the state of the module's `SD0` pin. Rather than hardcoding one address and leaving a silent failure mode for the

other variant, `initBME()` attempts 0x76 first and automatically retries 0x77 before reporting failure, removing an entire class of "sensor not found" support requests.

3. Schematic and Circuit

Schematic

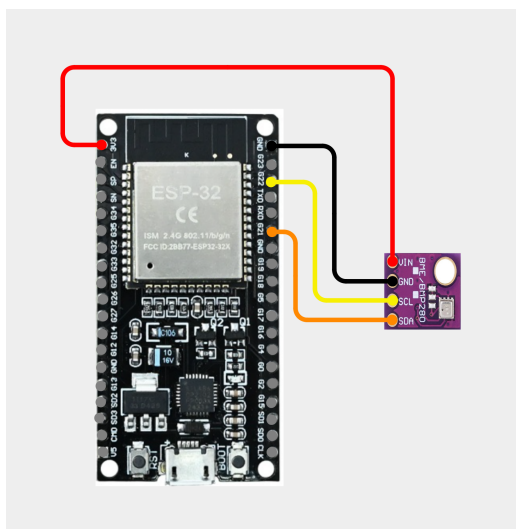


Figure 1: Wiring schematic

Assembled Circuit

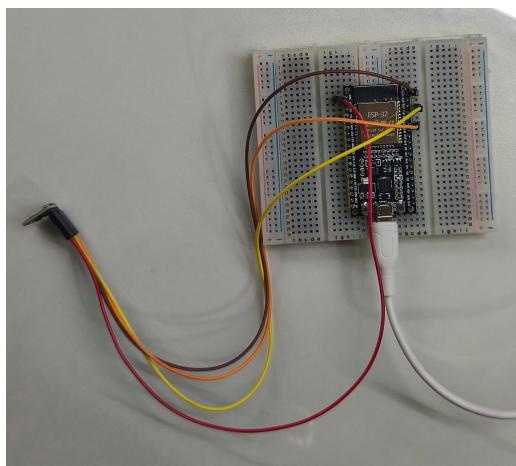


Figure 2: Photo of the assembled breadboard circuit under test

4. Software Architecture

The firmware is organized into three files, each with a single, well-defined responsibility – a deliberate separation even though the project does not use full C++ classes throughout:

File	Style	Responsibility
<code>config.h</code>	<code>#define</code> constants	Every WiFi credential, I2C pin, timing interval, and colour-zone threshold in the project. No other file contains a magic number – one place to retune the system.
<code>dashboard.html.h</code>	PROGMEM string	The entire single-page dashboard (HTML/CSS/JS) stored in flash, with <code>__TOKEN__</code> placeholders for the threshold values so the browser-side gauge logic and the firmware's threshold logic can never drift apart.
<code>main.cpp</code>	Application logic	Sensor polling, CSV logging, NVS persistence, WiFi connection management, and the HTTP route handlers.

Non-blocking, timer-based main loop

Every periodic action – sensor polling every `SENSOR_READ_INTERVAL_MS` and CSV logging every `CSV_LOG_INTERVAL_MS` – is implemented as a `millis()` comparison against a last-run timestamp, never a `delay()`. This matters specifically because the HTTP server (`ESPAsyncWebServer`) must keep answering requests – including a client mid-download of the CSV file – while the sensor loop is also running; a blocking `delay()` anywhere in `loop()` would stall every pending HTTP request for its duration.

Why `ESPAsyncWebServer` instead of the synchronous `WebServer` library

The synchronous `WebServer` library handles one request at a time and blocks the calling code until that request completes. For a small JSON poll this is harmless, but the `/download` route can be serving a CSV file of up to `CSV_MAX_SIZE_BYTES` (256 KB) at once – long enough, on a slow client connection, to noticeably delay the next `/data` poll if the server were synchronous. `ESPAsyncWebServer` handles connections in the background against the event loop, so a large CSV download in progress does not stall the live dashboard for other clients.

Single source of truth for thresholds

The colour-coded gauge zones (green/yellow/red) must agree between two places that would otherwise drift independently: the C++ constants in `config.h` and the JavaScript that paints the browser-side gauge arcs. Rather than maintaining the same six numbers by hand in two languages, `buildDashboardHtml()` performs a one-time `String::replace()` substitution of every `__TOKEN__` placeholder in the stored HTML with the corresponding `config.h` value at boot, producing `dashboardResolved` once and serving that resolved string on every subsequent `/` request. Changing a threshold is now a one-line edit in `config.h`, with both the firmware logic and the on-screen gauges updating automatically on the next flash.

5. Persistent Storage & Data Logging Architecture

The project uses **two independent flash-backed storage mechanisms**, deliberately chosen for two different jobs rather than overloading one system to do both.

5.1 LittleFS – append-only historical log

Every `CSV_LOG_INTERVAL_MS` (10s), a row of `timestamp,temperature,humidity,pressure` is appended to `/data.csv` on LittleFS. This is the durable, human-readable history intended for export and analysis (Section). To prevent this file from growing without bound over long unattended deployments, `appendCsv()` checks the file's size before each write and, if it exceeds `CSV_MAX_SIZE_BYTES`, deletes and recreates it with a fresh header – a simple rotation strategy appropriate for a single-file course project, at the cost of losing the oldest history rather than archiving it.

5.2 NVS (Preferences) – last-known-value persistence

The second requirement – surviving a power cycle without the dashboard going blank – is deliberately **not** implemented by re-parsing the tail of `data.csv` on boot. Instead, every successful CSV log of a *fresh* (non-stale) reading also calls `savePersistedReading()`, which writes the current temperature, humidity, pressure, and timestamp into a dedicated NVS namespace ("`envmon`") via the **Preferences** library.

NVS Key	Type	Purpose
<code>has</code>	bool	Whether a persisted reading exists at all (false on a truly first boot)
<code>temp / hum / pres</code>	float	The last known sensor values
<code>epoch</code>	unsigned long	Unix time of that reading, if NTP was synced when it was saved
<code>ts</code>	String	Human-readable timestamp string, for display even without NTP

On boot, `loadPersistedReading()` runs *before* WiFi or the sensor are even initialized, so the dashboard has something meaningful to show the instant the web server comes up – marked internally with `dataIsStale = true` until the very first fresh `pollSensor()` succeeds, at which point it flips to `false` and the cycle continues.

Why NVS instead of parsing the CSV's last line

Three concrete reasons drove this choice over the alternative (opening `data.csv` and reading its final row at boot):

1. **Boot latency is independent of log size.** Reading the last line of a file that may have grown close to `CSV_MAX_SIZE_BYTES` means either scanning the whole file line-by-line or implementing reverse-seek logic – **Preferences** is a fixed-cost key/value read regardless of how long the station has been logging.
2. **Decoupled from CSV rotation.** If `appendCsv()` has just rotated (deleted and recreated) the log file at the exact moment of a reboot, a "read the last CSV line" strategy could find an empty file. NVS is written independently of that rotation logic.
3. **NVS is designed for exactly this access pattern.** The underlying ESP-IDF NVS layer implements wear-levelling for small, frequently-rewritten key/value data, whereas LittleFS is optimized

for the CSV's append-heavy, larger-file access pattern. Using each storage system for the job it is designed for is preferable to making one do both.

5.3 Why flash survives power loss – and why it's the wrong tool for constant logging

Both `data.csv` (LittleFS) and the NVS reading in Section live on the same physical chip: the ESP32's onboard SPI NOR flash. Understanding what that chip is, what it is actually for, and what it costs to write to repeatedly explains both why persistence was possible at all and why it needed to be handled deliberately rather than treated as a free upgrade over RAM.

Why the data survives de-energizing

RAM (where `temp`, `hum`, `pres`, and every other plain variable in `main.cpp` live) is *volatile* – it stores a bit as an electrical charge on a capacitor that needs continuous power to refresh, and loses that state within microseconds of power loss. This is precisely why those variables reset to `NAN` on every reboot and why the dashboard would otherwise go blank until the next sensor sample. Flash memory stores each bit instead as a charge trapped on an electrically isolated *floating gate* inside the transistor itself. With no path for that charge to leak away, it remains in place with the power completely removed – the same physical mechanism used by a USB flash drive or SD card, and the same mechanism that lets the ESP32 boot the exact same firmware image every time it is powered on. Persisting the last reading and the CSV log is, in that sense, not a special feature bolted onto the device – it is the same non-volatility the chip already needed to remember its own program.

What this flash chip is actually there for

The SPI flash chip's primary job on an ESP32 is to hold the compiled firmware image, the bootloader, and the partition table – the things that must survive being unplugged, but that are written once (at flash time) and then read many thousands of times over the device's life without ever being rewritten during normal operation. LittleFS and NVS are carved out of *spare* space in that same partition table specifically so a sketch can also stash a modest amount of its own data there. The chip was not designed with "write a new record every ten seconds, indefinitely" as its primary use case – that access pattern is closer to how RAM or a purpose-built data-logging medium (e.g. an SD card) is normally used.

Why writing to it constantly is inefficient

Three properties of NOR flash make it a comparatively poor fit for the CSV's continuous append pattern, even though it works correctly:

1. **Erase-before-write granularity.** Flash cannot flip a bit from 0 back to 1 in place – an entire block (typically 4KB) must be erased before any byte within it can be rewritten. LittleFS hides this behind its normal file API, but the underlying operation is still a block erase/rewrite, not a cheap in-place byte update the way RAM or a spinning disk sector would be.
2. **Finite write/erase endurance.** Each flash block tolerates only a limited number of erase/write cycles before it becomes unreliable – typically on the order of 10^4 – 10^5 cycles for commodity NOR flash, several orders of magnitude below RAM, which has no such wear-out mechanism at all. LittleFS's wear-levelling spreads writes across blocks to delay this, but it manages the problem rather than eliminating it.
3. **Per-operation latency.** A flash write/erase takes on the order of milliseconds, versus nanoseconds for a RAM write – roughly a million-fold difference. Every call to `appendCsv()` is a small but real

blocking pause in `loop()`, which is tolerable at a 10-second cadence but would not scale to a much higher logging frequency without noticeable impact on the async server's responsiveness.

This is exactly why `CSV_MAX_SIZE_BYTES` (256 KB) is a *self-imposed* ceiling chosen well below the flash partition's actual physical capacity – it exists to bound how often the endurance and latency costs above are paid, not because the chip is nearly full.

Capacity math: how long until the log rotates?

Using the firmware's actual row format (`timestamp,temperature,humidity,pressure\n`) against a real logged row:

Quantity	Value
Example logged row	2026-07-28 07:38:38,25.11,38.67,895.20
Bytes per row (incl. newline)	39 bytes
Log interval (<code>CSV_LOG_INTERVAL_MS</code>)	10 s $\Rightarrow$ 8,640 rows/day
Rotation ceiling (<code>CSV_MAX_SIZE_BYTES</code>)	262,144 bytes (256 KB)
Rows until rotation	$262,144 \div 39 \approx 6,721$ rows
Time until the log fills and rotates	$6,721 \times 10\text{s} \approx 67,210\text{s} \approx 18.7\text{hours}$
Rotations per 24-hour day at this rate	≈ 1.3

In practice this means the on-device CSV history currently spans **under a day** before `appendCsv()`'s rotation logic deletes and restarts the file (Section 4). For a deployment that needs longer retained history, the practical options are: increase `CSV_MAX_SIZE_BYTES` (trading more flash wear/latency for longer retention), lower the logging frequency, or – the more scalable fix – export/download the CSV periodically before it rotates, or move long-term storage off the ESP32 entirely (an SD card or a remote server), which is exactly the kind of medium flash on this chip is not optimized to replace.

6. Web Dashboard & Connectivity

HTTP routes

Route	Purpose
GET /	Serves the resolved dashboard HTML/CSS/JS (single page, no reloads needed)
GET /data	Returns the latest reading as JSON, including a stale flag and an appropriate secondsAgo figure computed differently depending on whether the value came from a live poll or NVS
GET /status	Returns current WiFi mode (STA/AP), SSID, IP address, signal strength (RSSI), and uptime, for the dashboard's connection indicator
GET /download	Streams <code>/data.csv</code> from LittleFS with a Content-Disposition: attachment header, triggering a real browser file download rather than displaying raw text

Instrument-panel gauge design

Rather than plain numeric "stat cards," each metric is rendered as a semicircular dial with the green/yellow/red threshold zones drawn as actual coloured arc segments (a CSS `conic-gradient`, masked to a ring) and a needle that rotates live to the current value. The colour zones are the real configured thresholds, not decorative – the same numbers injected from `config.h` at boot (Section).

Connection-status feedback

The dashboard distinguishes three situations that are easy to conflate but mean very different things to a user standing in front of the device:

- **Live** (green dot) – the browser reached the ESP32 and the latest reading is fresh.
- **Stale / cached** (amber dot) – the browser reached the ESP32, but the reading currently being shown was restored from NVS after a reboot and no fresh sample has landed yet.
- **Connection lost** (red dot) – the browser's `fetch()` to `/data` itself failed, meaning the ESP32 is unreachable (powered off, out of range, or blocked at the network layer – see Section).

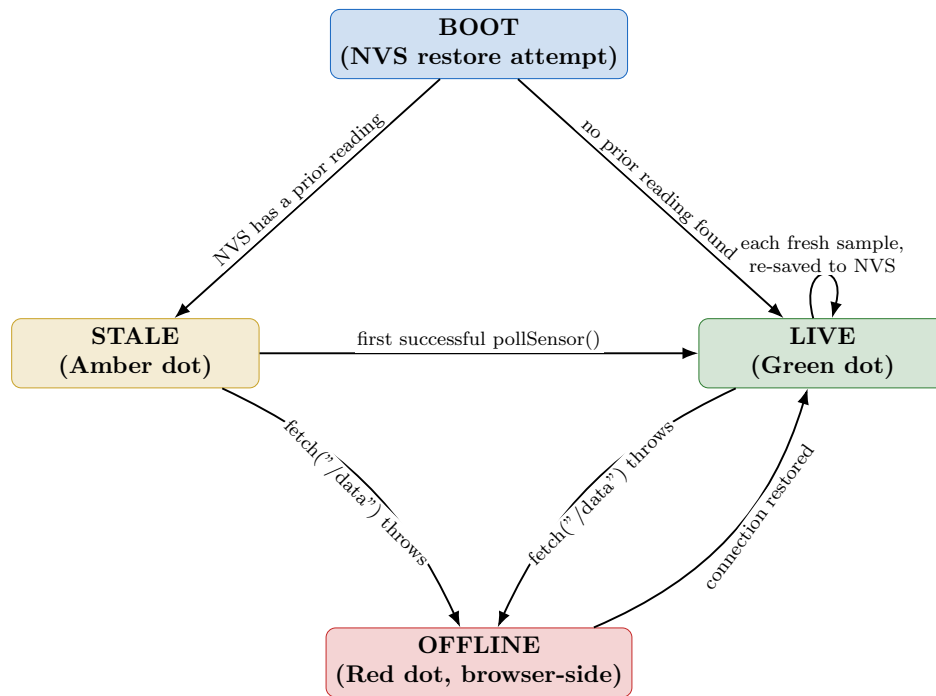
This distinction exists precisely because, during development, a "disconnected-looking" dashboard turned out to have two entirely different causes on two different occasions (Section), and a single generic "offline" indicator would have hidden which one it was.

WiFi station/AP fallback

`connectWifi()` attempts to join the configured network (`WIFI_SSID`) for up to `WIFI_CONNECT_TIMEOUT_MS`; on success it also starts NTP sync. If the timeout is reached, the ESP32 falls back to hosting its own access point (`AP_SSID`) so the dashboard is reachable even with no home/lab WiFi available at all.

7. Data-Freshness State Diagram

The dashboard's connection dot (Section 6) is the visible surface of an underlying state machine spanning both firmware and browser: what data is being shown, and how it got there.



Reading the diagram

The **BOOT** state is firmware-side and resolves in milliseconds. **STALE** and **LIVE** are also firmware-side (the `dataIsStale` flag), but are only ever *observed* by the browser through `/data`'s `stale` field. **OFFLINE** is purely a browser-side conclusion – the firmware has no concept of "offline" with respect to itself; it simply continues sampling and logging regardless of whether any browser is currently polling it.

8. Troubleshooting & Challenges

8.1 Isolating the sensor before building anything else

Before writing any WiFi, web server, or logging code, a minimal standalone sketch was flashed that only initialized the BME280 and printed readings to Serial every two seconds, with an I2C bus scanner as a fallback if the sensor did not answer at either expected address. This is the same isolate-the-smallest-testable-subsystem approach used in embedded debugging generally: with the sensor confirmed working in complete isolation, any later problem could be attributed to the WiFi/web/storage layers rather than re-litigating whether the wiring itself was ever suspect.

8.2 Arduino IDE vs. PlatformIO toolchain differences

Two separate compilation errors surfaced only after moving the project from the Arduino IDE to PlatformIO, both stemming from real differences between the two build systems rather than bugs in the code itself.

Missing library dependency. PlatformIO does not consult the Arduino IDE's globally-installed Library Manager; every dependency must be declared explicitly in `platformio.ini`'s `lib_deps`. The first build failed with `Adafruit_Sensor.h: No such file or directory` until `Adafruit BME280 Library` and `Adafruit Unified Sensor` were added there.

Missing function prototypes. A second build failed with `'scanI2C' was not declared in this scope`, even though the function was defined later in the same file and called correctly by name. The Arduino IDE silently auto-generates forward declarations for every function in a `.ino` file before compiling; PlatformIO compiles `main.cpp` as a plain C++ translation unit and performs no such preprocessing. The fix was adding explicit forward declarations for every function used before its own definition – visible at the top of `main.cpp` in the final firmware (Section).

8.3 Dashboard unreachable despite the ESP32 confirmed running

After the firmware had already been verified against the checklist – Serial Monitor showed the sensor initializing, WiFi connecting, NTP syncing to real calendar timestamps, and CSV rows being appended every ten seconds – the browser still reported `ERR.CONNECTION_TIMED_OUT` when loading the dashboard's IP.

Diagnosis. A connection *timeout* (as opposed to `ERR.CONNECTION_REFUSED` or a DNS failure) specifically means the request was sent but no response – in either direction – was ever received, which is a different failure signature from "the server isn't running." Because the Serial log already proved the HTTP server had started and the ESP32 held a valid IP, the fault was localized to the network path between the laptop and the device rather than the firmware. A simple `ping` to the ESP32's IP from the laptop was used to confirm this independently of the browser. Institutional/campus WiFi networks frequently enable client (AP) isolation, which deliberately blocks device-to-device traffic on the same network even though both devices retain internet access individually – explaining why the ESP32 could reach NTP servers while the laptop could not reach the ESP32.

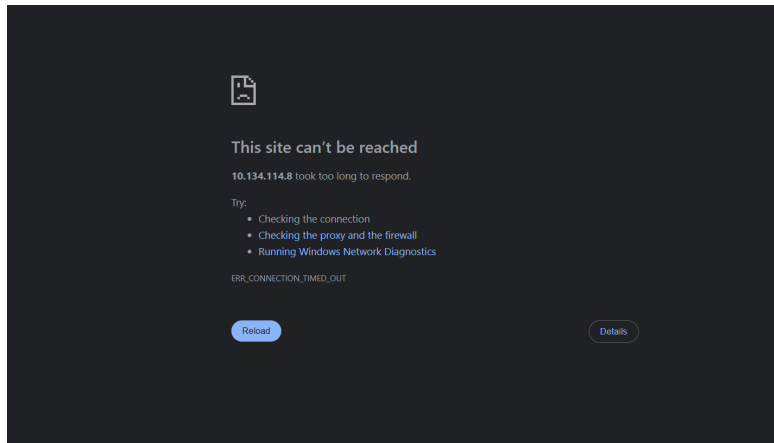


Figure 3: Browser ERR_CONNECTION_TIMED_OUT screenshot alongside the Serial Monitor proving the server was in fact running

8.4 Pressure threshold miscalibration for local altitude

The default warning/critical thresholds for barometric pressure were initially set around a sea-level baseline (approximately 1013 hPa). Once real sensor data was logged, the station consistently read approximately 895 hPa – BME280 pressure readings are absolute *station* pressure, not sea-level-normalized, and 895 hPa is the expected station pressure at this location's altitude. Left uncorrected, the pressure gauge would have shown a permanent, meaningless "critical" state on every boot. The green/yellow/red bounds in `config.h` (and, via the token-substitution mechanism, the browser's gauge) were rebased around the observed local baseline (880–910 hPa green, 860–930 hPa yellow) rather than a generic default.

9. Professional-Grade Additions

1. **Non-blocking main loop.** No `delay()` calls gate sensor polling or CSV logging; both are `millis()`-timed, keeping the async HTTP server fully responsive at all times.
2. **Dual-tier persistence strategy.** LittleFS for durable, exportable history; NVS for fast, rotation-independent last-known-value recovery (Section) – two systems, each matched to the access pattern it's actually good at.
3. **Automatic WiFi station/AP fallback.** The dashboard is reachable whether or not a configured network is available, without any manual reconfiguration step.
4. **Single source of truth for thresholds.** Boot-time token substitution guarantees the firmware's colour-zone logic and the browser's gauge rendering can never silently drift apart.
5. **CSV log rotation.** A size check before every append prevents unbounded flash growth on long-running deployments.
6. **Three-state connection feedback (Live / Stale / Offline)** instead of a single binary "connected" indicator, so the two very different failure modes uncovered in Section are visually distinguishable to whoever is standing in front of the dashboard.

7. **Graceful timestamp fallback.** `currentTimestamp()` returns a real calendar date/time once NTP has synced, or a `boot+N`s relative marker if it hasn't (e.g. pure AP mode with no internet) – logging never blocks waiting on a network that may not exist.

10. Results & Verification

Sensor accuracy and stability

The standalone test sketch (Section 8.1) confirmed stable readings (approximately 24–25 °C, 38–40% RH, 895 hPa) with no NaN glitches across a multi-minute observation window, before any networking or logging code was introduced – establishing a known-good sensor baseline against which later full-system readings could be compared.

Dashboard verification

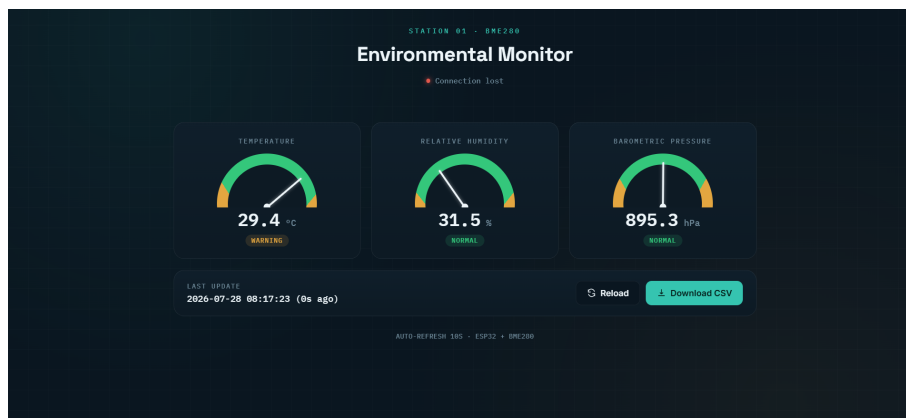


Figure 4: Dashboard screenshot when everything is fine

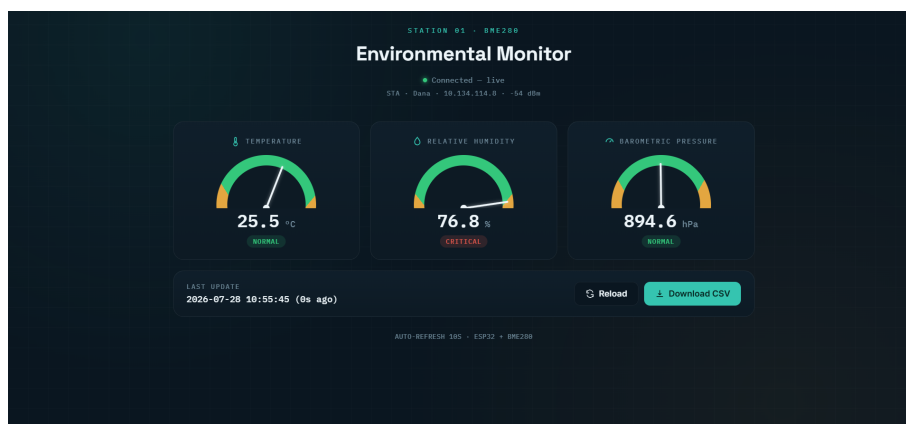


Figure 5: Dashboard screenshot – one gauge deliberately forced into a "Warning" or "Critical" state to demonstrate the colour-zone logic

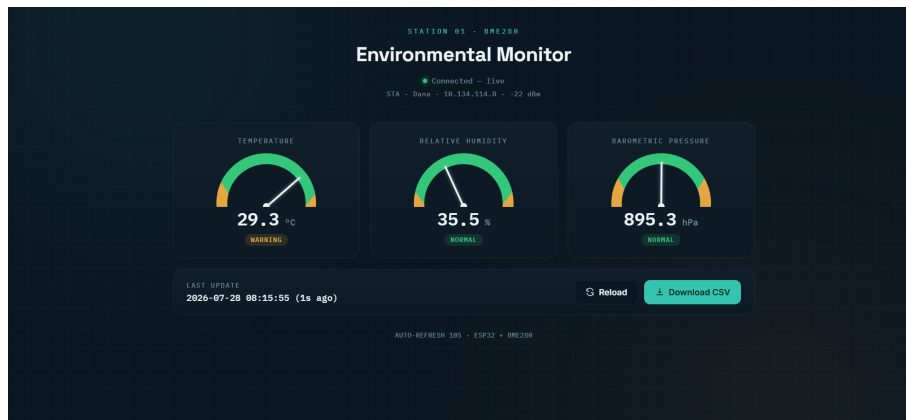


Figure 6: Dashboard screenshot showing that ESP32 is disconnected

CSV log verification

7/28/2026 8:23	27.72	34.37	895.28	
7/28/2026 8:24	27.79	33.05	895.31	
7/28/2026 8:24	27.88	33.34	895.31	
7/28/2026 8:24	27.86	32.92	895.28	
7/28/2026 8:24	27.87	32.95	895.29	
7/28/2026 8:24	27.89	33.38	895.28	
7/28/2026 8:24	27.89	32.99	895.3	
7/28/2026 8:25	27.9	32.65	895.3	

Figure 7: Downloaded sensor_log.csv opened directly in Excel, columns split automatically

11. Bill of Materials

Item	Qty	Notes
ESP32 DevKit development board	1	Main controller, WiFi + web server host
BME280 breakout module (I2C)	1	Temperature / humidity / pressure sensor, 3.3V logic
Breadboard + jumper wires	—	—
USB cable	1	Programming and power

12. What I Learned

A "connected/disconnected" indicator can hide two different problems

The single most valuable lesson from this build was that a browser timeout and a firmware fault look identical from the outside but require completely different fixes. Verifying the firmware's own Serial output *before* assuming the network path was at fault avoided misdiagnosing a network-isolation issue as a code bug (Section).

Toolchains are not interchangeable black boxes

Moving the exact same source files from the Arduino IDE to PlatformIO surfaced two build failures that had nothing to do with the code's correctness and everything to do with what each toolchain does or does not do automatically (dependency resolution, prototype generation). Understanding *why* a toolchain switch changes build behaviour was more useful than treating the fix as a one-off incantation.

Calibration against real data beats assumed defaults

The pressure threshold issue (Section 8.4) only became visible once real sensor data was logged – a reminder that environmental thresholds copied from a generic reference are a starting point, not a finished configuration, and should be checked against the device's actual deployment conditions.

Other takeaways

- Choosing the right storage system for each access pattern (NVS for small/frequent, LittleFS for append-heavy) avoided compromises either system would have forced individually.
- An async web server is worth the extra dependency the moment any route can plausibly take longer than a normal request/response cycle (here, CSV downloads).
- A single source of truth for shared constants (thresholds injected into the HTML at boot) removes an entire category of "the firmware and the UI disagree" bugs before they can occur.

13. Future Improvements

- **Configurable thresholds from the dashboard itself**, persisted to NVS alongside the last reading, removing the need to reflash for a threshold change.
- **CSV archiving instead of rotation-by-deletion** – e.g. rolling the oldest data into a secondary file rather than discarding it once `CSV_MAX_SIZE_BYTES` is reached.
- **HTTPS / basic auth** on the dashboard and download routes, relevant if this were ever deployed outside a trusted local network.
- **A second sensor node** reporting to the same dashboard, which the existing route structure (`/data`, `/status`) could be extended to support with minimal redesign.